Apache Spark vs. Hadoop MapReduce: A Comparative Analysis of In-Memory Processing Performance for Big Data Workloads

Ling Yu Qian
Faculty of Computing
Universiti Teknologi Malaysia
Johor Bahru, Malaysia
A23CS0301
SECP3133-High Performance Data Processing[

***Abstract*—The exponential increase of digital data has made conventional batch-processing systems insufficient to the needs of applications that are data-intensive. Hadoop MapReduce, which was the paradigm of large-scale distributed processing of data, has serious performance drawbacks due to its use of disk-based intermediate storage between computation phases, leading to large input/output overhead and excessive latency. To overcome these weaknesses, Apache Spark, a single in-memory distributed computing platform, was developed, with its Resilient Distributed Dataset (RDD) abstraction, which allows data to be stored in memory between stages of computation. The paper will compare and contrast Apache Spark and Hadoop MapReduce on various performance and usability criteria such as processing speed, fault tolerance, programming model, and mixed workloads. An example of a real-world case study of Netflix shows how Spark can be scaled to petabytes to personalise content recommendation. As can be seen in the comparative analysis, Spark can execute up to 100 times faster iterative machine learning workloads and is also the only system that provides native support of real-time streaming, graph processing, and SQL queries in a single unified system. Nevertheless, Spark has more memory demands, which are cost and resource challenges in memory-limited setups. The conclusion of this paper is that Apache Spark is a disruptive innovation in distributed data processing, but its implementation should be considered with respect to limitations of the infrastructure and the peculiarities of workload.**

***Index Terms*—Apache Spark, distributed computing, in-memory processing, big data analytics, performance benchmarking**

I. Introduction

The contemporary digital environment can be described by the fact that there is more data created by social networks, Internet of Things (IoT), financial systems, and science tools than ever before. The dynamic expansion of digital data has made conventional centralised computing to be insufficient, and the organisations have been forced to use distributed processing paradigms that can process datasets of petabyte size (Salloum et al., 2016). This has transformed processing, analysis and extraction of actionable insights of these large datasets into an acute competitive and operational necessity among organisations in all industry sectors.

The Hadoop MapReduce appeared in the middle of the 2000s as the base of the distributed large-scale data processing. MapReduce was introduced by Dean and Ghemawat (2008) at Google and offered a fault-tolerant, scalable architecture of parallelization of computations on groups of commodity hardware. Although widely used, MapReduce has fundamental performance constraints inherent to its disk-centric architecture: data between computation phases must be stored and read between phases in the Hadoop Distributed File System (HDFS), which introduces a great deal of input/output overhead and makes MapReduce ineffective in supporting iterative algorithms and real-time workloads, due to the repeated disk reads and writes required between each computation phase, which introduce significant latency.

To address the weaknesses of MapReduce, Apache Spark was created at the AMPLab in the University of California, Berkeley (Zaharia et al., 2012) in order to eliminate the weaknesses of MapReduce. Spark supports in-memory computation by the RDD abstraction, which drastically minimizes the disk I/O overhead that limits the performance of MapReduce. Moreover, Spark offers a single platform that augments batch processing, real-time streaming, machine learning, and graph computation in a single platform, therefore, significantly lowering the complexity of the operation of multiple specialised systems.

Nevertheless, there is one inherent cost associated with these benefits, namely, the performance of Spark is highly dependent on the amount of memory available, and organisations that implement it without taking due consideration of their infrastructure may run the risk of having to pay a lot of overhead without equivalent returns.

The paper compares and contrasts Apache Spark and Hadoop MapReduce on the basis of major performance and architectural aspects. This aims to develop a systematic evidence-based view of the benefits, shortcomings and the right application of Spark recognising that technological superiority is always contextual, never absolute. The rest of the paper is organized in the following way: Section II gives a technical description of the architecture of Apache

Spark; Section III compares and contrasts it with Hadoop MapReduce; Section IV contains a case study; and Section V draws a conclusion and ideas on future research.

II. Apache Spark: Architecture and Core Concepts

A. Overview

Apache Spark is an open-source, generalized analytics engine of large-scale data processing, which is fast, easy-to-use, and generalized (Zaharia et al., 2016). Fundamentally, Spark offers an in-memory computing paradigm that allows data to remain in memory across computation phases and not be written back to disk repeatedly, which is a paradigm shift with MapReduce.

B. Resilient Distributed Datasets (RDDs)

The basic abstraction of data in Spark is the Resilient Distributed Dataset (RDD), which was proposed by Zaharia et al. (2012). An RDD is an unchanging, partitioned set of records, which can be acted upon concurrently in a cluster. RDDs are resilient, in the sense that they record the history of operations performed to convert them to source data, allowing lost partitions to be automatically recovered without needing to replicate entire data. This fault tolerance based on lineage is computationally and storage efficient as compared to HDFS replication. RDDs support lazy transformations (e.g., map, filter) and eager actions (e.g., count, save), enabling Spark to optimise full computation pipelines before execution (Zaharia et al., 2013). However, it is notable that this beauty is memory-constrained, the datasets that are bigger than the available RAM will need to be spilled to disk, which is somewhat countering the very benefit that RDDs are intended to offer. Even the lineage graph in itself may be a liability in such situations, since it is expensive to re-compute long chains of transformations when they fail.

C. Cluster Architecture

Spark is based on a master-worker architecture. As illustrated in Figure 1, the User application is executed by the Driver Programme which transforms the application logic into a Directed Acyclic Graph (DAG) of tasks. The Cluster Manager (it may be Apache YARN, Mesos or Kubernetes) distributes compute resources on the cluster. Executors are JVM processes running on Worker Nodes and execute single tasks and store data in memory. Such an architecture allows managing resources on a fine scale and dynamically scheduling tasks (Zaharia et al., 2016).

D. Spark Ecosystem Components

Spark is an ecosystem with a rich set of integrated libraries that are overlaid onto the core engine, such as Spark SQL, Spark Streaming, MLlib, and GraphX. Spark SQL supports querying structured data with a DataFrame API and SQL syntax and the Catalyst query optimiser (Armbrust et al., 2015). Spark Structured and Spark streaming enable near real-time processing of data through micro-batch processing.

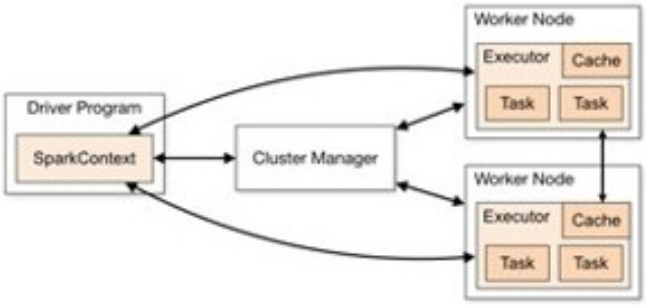

Fig. 1. Apache Spark Cluster Architecture (modified after Zaharia et al., 2016).

MLlib offers scalable machine learning algorithms such as classification, regression, clustering, and collaborative filtering (Meng et al., 2016). GraphX is a graph-parallel computer. This unification into a single framework greatly lowers operational overheads against implementing a number of specialised systems (Salloum et al., 2016).

III. Comparative Analysis: Apache Spark vs. Hadoop MapReduce

Before Apache Spark, the most popular system of distributed large-scale data processing was Hadoop MapReduce. Nonetheless, MapReduce necessitated intermediate results to be stored to disk between processing steps and this incurs high I/O overheads and latency (Dean and Ghemawat, 2008). Apache Spark overcame this basic shortcoming by using in-memory computing, which provided orders of magnitude better performance on iterative workloads. A systematic comparison is provided below with regard to major criteria.

The difference in performance of Spark versus MapReduce is the greatest in the case of iterative algorithms. Benchmarks by Zaharia et al. (2016) show that Spark can run iterative machine learning jobs up to 100 times faster, due to in-memory caching of intermediate datasets, which are reused across multiple passes of computation. The performance difference is less impressive, around 10 times faster, with batch processing workloads with no iteration (Salloum et al., 2016). What is not reflected by these benchmark figures, however, is the cost of infrastructure behind them. An increase in speed by 100x sounds impressive on paper but it assumes that there is enough RAM available to support condition that is hardly guaranteed in most real world deployments. Commodity hardware based organisations or organisations with limited budgets on cloud provisioning may discover that the cost of provisioning the memory-intensive Spark cluster is higher than the performance improvement, especially where the workload is not iterative in nature. The benchmark story in this respect can be deceptive: it is offering the ceiling performance of Spark without putting the floor conditions of the performance into context.

But there are trade-offs to performance benefits of Spark. Once the working dataset is larger than what can be held in RAM, Spark is compelled to spill the data to disk, significantly slowing down performance and to some extent

TABLE I
COMPARATIVE ANALYSIS - APACHE SPARK VS. HADOOP MAPREDUCE

Criteria	Hadoop MapReduce	Apache Spark
Processing Model	Disk-based; intermediate results are written to HDFS between each stage	In-memory computation using RDDs; disk I/O is minimised
Processing Speed	Slower due to multiple disk reads/writes; high latency on iterative jobs	Up to 100x faster on iterative workloads; 10x faster on batch workloads (Zaharia et al., 2016)
Programming API	Mainly Java; verbose with cumbersome boilerplate code	Supports Python, Scala, Java, R, SQL; concise high-level APIs
Fault Tolerance	Data replication between HDFS nodes; offline recovery by re-reading	RDD lineage graph; lost partitions recomputed from original transformations
Real-Time Streaming	Not supported natively; must integrate with Apache Storm separately	Spark Streaming and Structured support near real-time data processing
Machine Learning	External library (Apache Mahout); inefficient with iterative ML algorithms	Built-in MLlib; optimised for iterative ML with in-memory caching
Memory Requirement	Low; relies on disk storage and HDFS replication	Higher RAM required; performance decreases as dataset surpasses available memory (disk spill)
Best Use Case	Simple, large-scale batch processing where memory is constrained	Mixed workloads: batch, streaming, real-time pipelines, graph processing, and ML

nullifying its in-memory benefit. Spark clusters can have per-node RAM needs that can be very expensive to satisfy in memory-constrained environments, in comparison to MapReduce, which can execute on disk-heavy, memory-light commodity hardware (Shi et al., 2015).

Furthermore, Spark's high-level APIs are much easier to write than the verbose Java-based programming model of MapReduce. Python (PySpark), Scala, R, and SQL interfaces enable data scientists and analysts who might not be knowledgeable about Java to enter the field dramatically less expensively (Armbrust et al., 2015). DataFrame and Dataset API in Spark 2.0 also enhance developer productivity with strongly typed schema-aware abstractions, auto-optimised queries with the Catalyst engine.

IV. REAL-WORLD CASE STUDY: NETFLIX

The most interesting example of the capabilities of Apache Spark in petabyte scale is Netflix, the most successful subscription-based streaming service that has more than 200 million subscribers worldwide (Steck et al., 2021). Apache Spark is a fundamental element of the Netflix data analytics and personalisation platform which processes terabytes of user interaction data each day to drive its recommendation engine.

The recommendation system of Netflix uses viewing history, ratings, search-queries, and contextual cues of every subscriber to produce the individual content recommendations. The underlying collaborative filtering and content-based filtering algorithms which support these recommendations are iterative by nature, and necessitate repeated execution over large datasets which is the workload profile where Spark in-memory computing model is most advantageously deployed in comparison to MapReduce (Salloum et al., 2016).

Steck et al. (2021) report that Netflix uses the MLlib library of Spark to scale personalisation model training to allow quick experimentation with deep learning and collaborative filtering strategies on its entire subscriber base. Moreover, Spark Streaming helps Netflix to handle real-time events of user interactions, which can help the recommendation system to react dynamically to in-session viewing behaviour as opposed to basing it purely on historical models that are computed in batches (Zaharia et al., 2013). This real time and batch processing in one unified Spark system does away with the architectural complexity of having to maintain separate systems to support each processing paradigm.

The use of Apache Spark by Netflix shows that the framework is not only beneficial when used in a benchmark setting, but also when used in a mission-critical internet-scale production environment. However, it is worth noting that Netflix is a unique situation that has the engineering resources, investment in infrastructure, and data volume to warrant the resource needs of Spark. To most organisations that do not have operations of the scale of the petition-based organisations, the example of Netflix is educative and not directly applicable. Instead of just the allure of industry precedence, the choice to go with Spark must be motivated by a candid evaluation of the nature of workload and resource limitations.

Conclusion This paper has provided a comparative analysis systematically of Apache Spark and Hadoop MapReduce in terms of architectural design, processing performance, programmability, fault tolerance, as well as the applicability to various workload types. This discussion has shown that Spark is a revolutionary improvement in distributed data processing, providing up to 100 times faster execution time on iterative workloads with in-memory RDD abstraction, a much more accessible and expressive programming model, and a single platform, which does not require multiple specialised frameworks to process batches, streams, machine learning and graph processing.

The case study of Netflix in real-life shows that the architectural benefits of Spark are directly reflected in tangible operational benefits at scale, such as the shorter model training latency and the capability to perform batch and real-time processing in the same coherent system. These abilities are becoming more and more vital as organisations attempt to gain a competitive edge using data as fast as business.

However, the increased memory needs of Spark are a real drawback in resource-limited or cost-sensitive deployment context. Workloads that are not iterative, e.g. simple batch workloads, may not be worth the infrastructure costs when operating on large datasets that might exceed available RAM, and the performance benefits of Spark are reduced in this case. The Hadoop MapReduce is still a feasible and cost-efficient option in that case, especially in situations where the deployment of Hadoop infrastructure already exists.

The performance properties of Spark in cloud-native and

serverless deployment settings should be explored in future research, where the cost limitations observed in this analysis might be counteracted by elastic memory provisioning. Also, the introduction of more recent distributed processing systems like Apache Flink, which provides actual event-at-a-time processing as opposed to micro-batch processing, should receive systematic comparison with Structured Streaming of Spark in their latency-sensitive applications. The further development of hardware accelerator, such as GPU-accelerated distributed computing through RAPIDS cuDF integration with Spark, is another promising way of improving distributed data processing performance much more effectively than can be done by in-memory CPU-based computation alone.

LOG BOOK

Student Name: LING YU QIAN
Matric number: A23CS0301
Course: SECP3133

TABLE II
LOG BOOK ENTRIES

Date	Search Keywords	AI Prompts Used	Task Description
5 April 2026	Apache Spark performance Scopus 2022	None	Searched Scopus and IEEE Xplore for peer-reviewed papers on Apache Spark performance. Found 14 relevant articles. Selected 10 to read in full.
7 April 2026	Spark vs Hadoop MapReduce benchmark comparison	Explain what in-memory processing means in simple terms	Read 6 papers and took detailed notes on key performance differences. Began drafting the comparison table with criteria.
9 April 2026	Apache Spark RDD architecture cluster distributed	None	Studied Spark cluster architecture (Driver, Executor, DAG Scheduler). Began writing Section II (Spark Overview and Architecture).
11 April 2026	Netflix Apache Spark recommendation system case study Scopus	Help me explain Netflix Spark use case in academic language	Found and read case study papers on Netflix's Spark deployment. Wrote Section IV (Real-World Case Study).
13 April 2026	Spark Hadoop fault tolerance comparison VLDB	None	Completed comparative analysis table (Table 1). Wrote Section III with in-text citations for all comparison points.
15 April 2026	Apache Spark limitations memory constraints streaming	None	Wrote Introduction and Conclusion sections. Ensured paper objective and future research directions are clearly stated.
17 April 2026	APA 7th edition journal citation format IEEE	Help me format this sentence	Proofread entire paper for formatting and grammar. Formatted all 10 references in APA 7th Edition.

REFERENCES

[1] M. Armbrust, R. S. Xin, C. Lian, Y. Huai, D. Liu, J. K. Bradley, X. Meng, T. Kaftan, M. J. Franklin, A. Ghodsi, and M. Zaharia, "Spark SQL: Relational data processing in Spark," in *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data (SIGMOD '15)*, pp. 1383-1394, 2015. https://doi.org/10.1145/2723372.2742797

[2] J. Dean and S. Ghemawat, "MapReduce: Simplified data processing on large clusters," *Communications of the ACM*, vol. 51, no. 1, pp. 107-113, 2008. https://doi.org/10.1145/1327452.1327492

[3] X. Meng et al., "MLlib: Machine learning in Apache Spark," *Journal of Machine Learning Research*, vol. 17, no. 34, pp. 1-7, 2016. https://jmlr.org/papers/v17/15-237.html

[4] S. Salloum, R. Dautov, X. Chen, P. X. Peng, and J. Z. Huang, "Big data analytics on Apache Spark," *International Journal of Data Science and Analytics*, vol. 1, no. 3-4, pp. 145-164, 2016. https://doi.org/10.1007/s41060-016-0027-9

[5] J. Shi, Y. Qiu, U. F. Minhas, L. Jiao, C. Wang, B. Reinwald, and F. Özcan, "Clash of the titans: MapReduce vs. Spark for large scale data analytics," *Proceedings of the VLDB Endowment*, vol. 8, no. 13, pp. 2110-2121, 2015. https://doi.org/10.14778/2831360.2831365

[6] H. Steck, L. Baltrunas, E. Elahi, D. Liang, Y. Raimond, and J. Basilico, "Deep learning for recommender systems: A Netflix case study," *AI Magazine*, vol. 42, no. 3, pp. 7-18, 2021. https://doi.org/10.1609/aimag.v42i3.18140

[7] M. Zaharia, M. Chowdhury, T. Das, A. Dave, J. Ma, M. McCauley, M. J. Franklin, S. Shenker, and I. Stoica, "Resilient distributed datasets: A fault-tolerant abstraction for in-memory cluster computing," in *Proceedings of the 9th USENIX Symposium on Networked Systems Design and Implementation (NSDI'12)*, pp. 15-28, 2012.

[8] M. Zaharia, T. Das, H. Li, T. Hunter, S. Shenker, and I. Stoica, "Discretized streams: Fault-tolerant streaming computation at scale," in *Proceedings of the 24th ACM Symposium on Operating Systems Principles (SOSP '13)*, pp. 423-438, 2013. https://doi.org/10.1145/2517349.2522737

[9] M. Zaharia et al., "Apache Spark: A unified engine for big data processing," *Communications of the ACM*, vol. 59, no. 11, pp. 56-65, 2016. https://doi.org/10.1145/2934664